

Entropy Noise Source for TRNG

Paul Scheidt

Revision

Version	Date	Author	Description
0.1	6. Nov. 2025	Paul Scheidt	First draft
0.5	13. Nov. 2025	Paul Scheidt	Added BIW extractor entropy compressor.
0.6	18. Nov. 2025	Paul Scheidt	Automatic ring detuning mechanism added.
0.7	11. Dec. 2025	Paul Scheidt	Minor updates
0.8	18. Dec. 2025	Paul Scheidt	Pasted in latest Register Memory Map derived from SystemRDL source. Clarified detune state transition trigger when health test failure is asserted. Transition happens only at assertion time (positive edge) and is “sticky” until next assertion.

Future Feature Enhancements

Feature	Description and Motivation
Double Number of Rings to 24	Double the entropy rate and improve attack resilience.
Increase number of sampling points	Harvest more entropy per ring thus increasing entropy rate.

Abstract

This document describes an entropy noise source component which will be integrated into a cryptographically secure True Random Number Generator (TRNG). The TRNG is a critical component required to support cryptographic operations used to govern chip security. Their attributes are specified in NIST SP800-90B and validated with FIPS 140-3 certification procedures.

This component will be implemented as a test vehicle in the Atlantis chip. In-house performance evaluation will prepare the final design shipped with OCH to be FIPS certifiable. This component will be implemented in the TSMC 12nm process used by Atlantis.

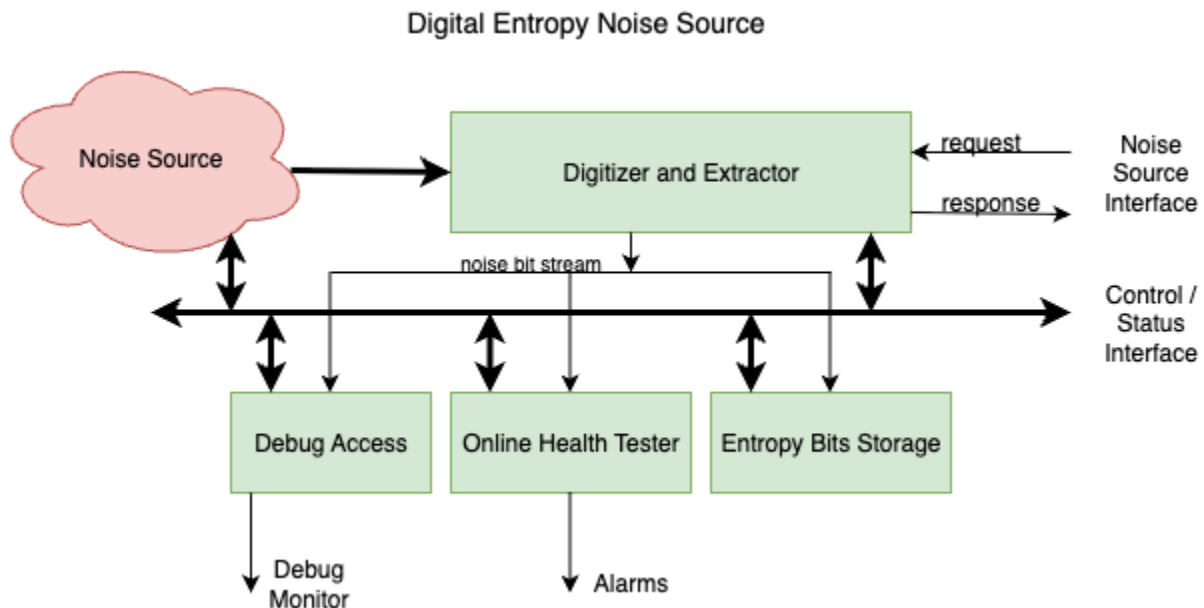
This document fully describes the functional design of the TRNG and also provides physical design (PD) implementation guidelines to ensure.

Introduction

The entropy source component is a random number generator which is based on multiple asynchronous ring oscillators sampled in such a way to generate random bits from the jitter noise and metastable latching behavior. It forms the basis of a True Random Number Generator (TRNG).

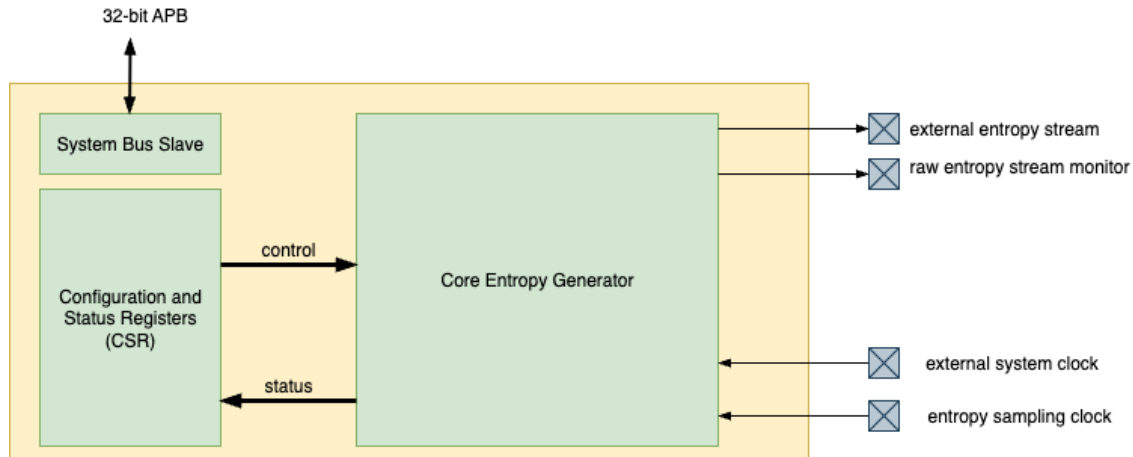
The generated stream is “raw” noise in the sense that there may still be some statistical biases present which need to be filtered out by an optional entropy conditioner post processing unit which would consume the output noise bits. NIST specifies a set of acceptable conditioners. Common conditioners are based on standard hash functions such as SHA256 and ASCON as well as standard keyed hash functions such as AES-CMAC and HMAC.

The goal of this test component is to assess the statistical properties of the raw entropy source, so we will not add a conditioner here.



The diagram below shows the entropy source component which implements the above concept. The client controls the component via the APB bus interface which provides access to a set of memory mapped configuration and status registers (CSRs) described later in this document. The raw entropy stream is connected to the (optional) downstream conditioning unit mentioned above as well as the health statistics tester, debug monitor and entropy bits storage.

The entropy source is designed according to guidelines set forth in the NIST SP800-90B specification and is planned to be ready for FIPS 140-2 and 140-3 certification. It will also be compliant with AIS31 statistical requirements specified by the BSI agency.

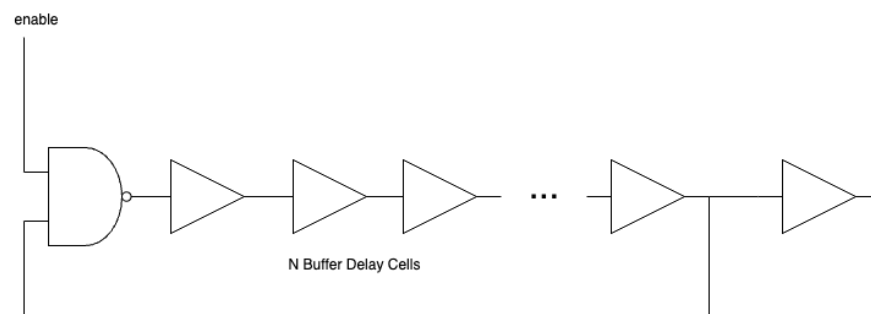


Noise Source

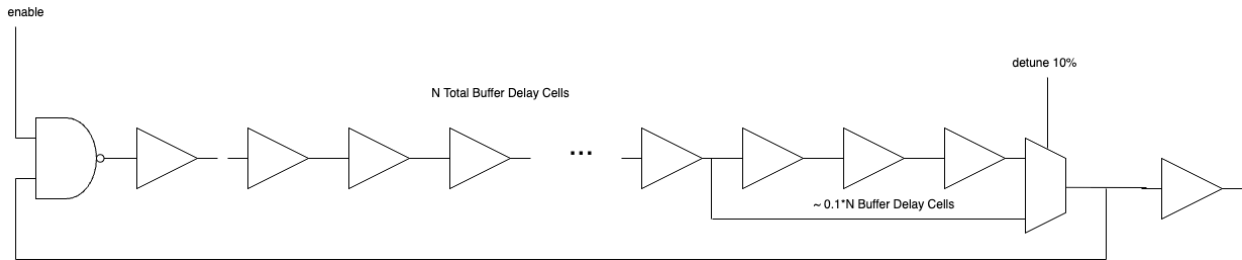
The entropy noise is generated by sampling a set of ring oscillators (RO). The lengths of the ROs are coprime (before detuning). The jitter in the ring oscillator along with the metastable sampling behavior is the source of the entropy.

Ring Oscillators (RO)

The basis of the entropy source is the ring oscillator (RO) shown below. It consists of a number of buffer cells along with an inverting NAND cell connected in an asynchronous ring. The RO will oscillate when the enable is asserted high. The oscillation period is dependent on the number of cells in the ring and the combinatorial propagation delay through each of those elements. The oscillation will naturally jitter and this is dependent on the noise on the power and ground rails as well as temperature fluctuations.



The RO frequency can be tuned by introducing a selector which can switch in or out additional inverters as shown below. In this design, each oscillator frequency can be adjusted by approximately 5-10% by switching in extra delay buffers.



The TSMC 12nm BUFFD0 cell is used as the main delay element. It has the following propagation delay properties (from the datasheet):

$t_{PLH} = 0.0161 + 3.7336 * C_{load}$ ps positive edge (low to high) propagation delay

$t_{PHL} = 0.0149 + 4.0475 * C_{load}$ ps negative edge (high to low) propagation delay

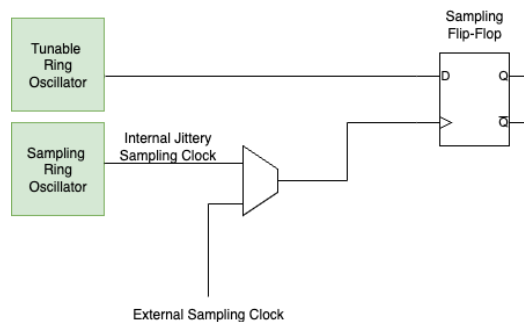
With the C_{load} being equal to the BUFFD0 input capacitance of 0.0004906 pF (nominal), we get an average nominal propagation delay of 17.408 ps. Thus, a ring oscillator running at nominally 1 GHz will hence require approximately 57 buffer cell stages. These numbers give us a starting point in determining the number of delay stages for each ring oscillator.

This design will have a set of 16 ring oscillators operating with a range of frequencies from approximately 0.7-4.5 GHz.

Sampling Clock and Ring Oscillators

Ideally, we would like to sample these sources at about 1/10th of the natural ring oscillator frequency. This corresponds to a range of approximately 100 to 450 MHz.

This component will have the option of using a single external sampling clock or an internally generated jittery clock generated by another set of ring oscillators.



The table below specifies the ring oscillator lengths and their resulting frequencies.

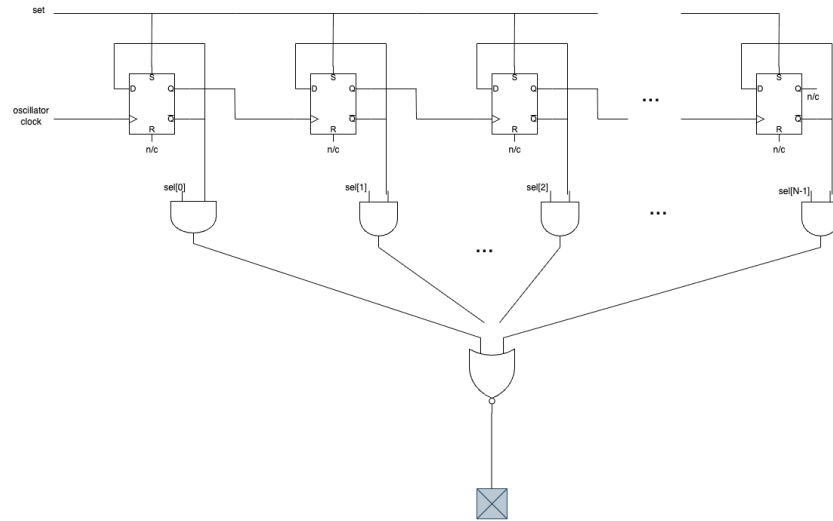
Note: the numbers below assume a pure buffer ring. However the rings also have one NAND gate and in the case of a detunable ring, one multiplexor in the path. Therefore, each of the entries for the number of buffers should be reduced by 2 or 3 buffer cells e.g. RO₀ uses 11 buffers plus one mux and one nand cell.

RO Instance	Number Buffers	Freq (MHz)	Detuned Number Buffers	RO Freq (MHz)	Sampling RO buffers	Sampling RO freq (MHz)	Freq Ratio
0	13	747	7	4418	131	438	10.1
1	17	2498	9	3379	173	332	10.2
2	19	1981	11	3023	191	300	10.1
3	23	1853	19	2479	229	250	9.99
4	29	1553	24	1980	293	196	10.1
5	31	1401	25	1853	311	184	10.0
6	37	1336	31	1552	373	154	10.0
7	41	1222	33	1401	409	140	10.0
8	43	1084	36	1335	431	133	10.0
9	47	974	39	1222	467	123	9.93
10	53	942	45	1083	523	109	9.94
11	59	857	49	973	593	96	10.1

Ring Oscillator Test and Debug Monitor

We will need to monitor the output signal of a ring oscillator on the chip output pad. This is hindered by its very high oscillation frequency. To deal with this, we use an asynchronous clock divider circuit (a ripple counter) to generate a series of lower frequency clock replicas divided by 2, 4, 8, ... $2^{**}N$ followed by a selector. The monitor thus has two programmable controls:

- Input select
- Clock divider select

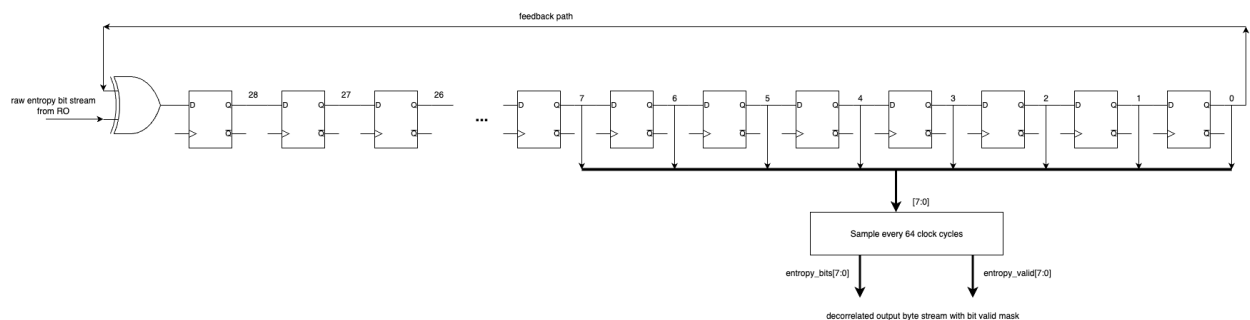


Serial Decorrelator

Ring oscillator entropy sources are prone to serial correlation which when combined results in reduced entropy. This is corrected using a serial decorrelator circuit.

The total raw entropy bit rate out of the collection of ring oscillators is approximately 4.179 Gbps. This is determined by adding all of the contributions of the 16 sampled ring oscillators. This will not be a full entropy rate and further processing to decorrelate will reduce the bit rate while increasing the entropy rate.

Decorrelation is done using an undersampled 29-deep XOR feedback shift register. Each RO output will feed one of these units. Bits from each RO sampler enter the shift register after XORing with the random bit produced 29 cycles earlier. The last 8 bits of the shift register are sampled every 64 cycles. The values 64 and 29 are coprime which guarantees that the input bit influence rotates around consecutive samples. It takes $29 \times 64 = 1856$ cycles for a specific input bit to affect all 8 of the output bits. This scheme reduces the output bit rate by a factor of 8 to approximately 522 Mbps.



Configuration Options

The default configuration of the decorrelator is that shown in the diagram above. We can provide some CSR configurable options to enable experimenting with different extractor schemes.

- Output sampling rate
 - Default is 64, but could be increased or decreased to change entropy and bit rates
- Shorter shift register chain with smaller number of sample bits
 - Length 7, sample 4 bits (with output sampling rate 16)
- Bypass decorrelator entirely
 - 8 bits output every 8 cycles, no xor feedback
- LFSR mode (Fibonacci topology)
 - Length 29 (default), with primitive polynomial $x^{29} + x^2 + 1$
 - Length 7, with primitive polynomial $x^7 + x^6 + 1$

Entropy Compressor

The combiner takes 12 input byte streams from the decorrelator / extractor block and compresses them into a single stream 32-bits wide.

The 12 byte streams are formed into four groups of 3. Each 3 byte streams are fed into a BIW extractor to produce a single byte stream. These four are combined into a 32-bit word which is input to the entropy storage FIFO.

The BIW extractor computes $y = a*b+c$ in a $GF(2^8)$ field using the primitive polynomial $x^8 + x^4 + x^3 + x + 1$.

Statistical Health Test Monitor

The RNG noise source is a potentially fragile device. The online statistical tests are used to confirm that the RNG noise source has not deviated from expected behavior and suffered a catastrophic failure. The health test monitors a set of metrics to determine this. These tests are not intended to measure actual entropy. That is left for offline tests.



The mission of this unit is to detect gross failures quickly with high probability so countermeasures such as restart or retune can be initiated rapidly. We also want to prevent a misbehaving TRNG from exporting low entropy bit streams downstream to the DRBG. It is also very important to minimize the probability of false positives, so that the TRNG is available with high probability i.e. prevent denial of service.

SP800-90B section 4.2 describes the three categories of health tests:

1. Start-up tests
 - a. These tests are run repeatedly after simulating a power-on scenario to detect bad startup behavior
2. Continuous tests

3. On-demand tests

- a. These can be the same as the start-up tests but run any time “on-demand”

SP800-90B section 4.4 recommends two health tests:

- Repetition test
 - Goal: detect failures that cause the noise source to become stuck on a single output value for a long period of time
- Adaptive proportion test
 - Goal: detect when one value has a higher probability of occurrence i.e. biased

The entropy source will implement the repetition and adaptive proportion tests. It will also implement the following extra test:

- Markov test
 - Goal: detect if there is any dependence between consecutive outcomes which would indicate that we do not have a memoryless source.

Repetition Test

This test has the goal of detecting the catastrophic “stuck at” state. It is defined as follows:

H = entropy estimate for noise source

$P(\max) = 2^{-H}$ is the maximum probability for any value

α = acceptable false positive probability

$C = 2 + \text{ceil}(-\log(\alpha)/H)$

Implementation

Count how many times we see a repetition. If $\text{count} \geq C$, then declare a failure.

Store the precomputed value C in a CSR configuration register as a threshold value.

Maintain the state of the previous noise source `last_sample` and a counter.

```
counter = 0
while(True): # loop continuously
    if (new_sample == last_sample) then:
        counter += 1
        if (counter >= C) then
            assert_failure = True
    else:
        last_sample = new_sample
        counter = 0
```

The `assert_failure` triggers an interrupt to be generated.

A good default starting point is having the threshold $C=21$, but of course this depends on expected entropy and acceptable false positive probability.

Adaptive Proportion Test (APT)

This tester measures the frequency of occurrence of a sample value and detects when one value becomes too common. It takes a particular sample value and counts the number of times this sample occurs within a window of $W-1$ samples. The algorithm details are described in section 4.4.2 of the NIST SP800-90B document and here is a summary.

$W = 1024$ (binary sources) or 512 (non-binary)

Based on the estimate of H (entropy/sample), W and an acceptable false positive rate α , we choose a cutoff value of C .

For every W samples, we restart the test. We take the first sample in the window and count how many times it reappears in the whole window. If it appears C or more times, then we declare a failure.

TODO: define the pass/fail threshold values to be programmed into configuration registers

Implementation

We require configuration registers to store C and W . In our case we have a non-binary source so choose $W = 512$. Maintain the state of the value being counted and the counter itself.

```
repetition_count = 0
while(True): # loop continuously
    last_sample = new_sample
    for sample_counter in range(W):
        if (new_sample == last_sample) then:
            repetition_count += 1
        If (repetition_count >= C):
            assert_failure = True
    repetition_count = 0
```

The `assert_failure` triggers an interrupt to be generated.

A good default starting point for the binary test is declaring a failure if $435 \leq \text{repetition_count} \leq 589$, but of course this depends on expected entropy and acceptable false positive probability.

Markov Test

BSI AIS31 B.1.1 specifies the Markov test. This test analyzes the ratio of switching a) 0 to 1 b) 1 to 0 c) 0 to 0 and d) 1 to 1 on consecutive bits.

Runs Test

As a stretch feature, we can replicate the FIPS 140-3 runs test in hardware. We maintain a set of counters that measures how many runs of 0s and 1s of particular lengths we've detected. Run lengths to be monitored correspond to FIPS requirements: 1, 2, 3, 4, 5 and 6. This is discussed in more detail later in this document.

There is one threshold configuration for each counter. If the threshold is exceeded, fire an interrupt.

NOTE: this feature is not implemented on the prototype component for Atlantis

Entropy Bits Storage

The decorrelated entropy stream is collected in a storage unit which is accessible by the host via the APB bus. The storage unit is a 32-bit wide and 64 deep FIFO. It can collect up to 1024 entropy noise bits from the digitizer for offload and post processing. The FIFO is 32-bits wide with 32-bit memory mapped read access via the APB slave interface.

The tests are performed offline, not on chip. We only need to provide a facility to capture the necessary amount of continuously generated data bits. Since FIPS tests generally require 20000 bits, we will need the on-chip host processor to drain this FIFO and store entropy bits at least as fast as they are being generated by the noise source. Assume that the host processor is running at 1 GHz and we can perform continuous APB read transactions with 2 cycles per transaction. The reading rate is thus approximately $0.5 \text{ G} * 32 = 16 \text{ Gbps}$. Assume a worst case noise sampling rate of 500 MHz. We estimate that our entropy noise source bit rate is approximately $16 \text{ streams} * 8/64 * 1/8 * 500 \text{ MHz} = 125 \text{ Mbps}$. This gives us two orders of magnitude margin.

For background, FIPS 140-3 standard specifies the following statistical test procedures on the collected entropy noise bits. We will perform these tests offline on data collected from lab measurements once we have silicon.

- Poker
 - 20k bit block is divided into 5000 consecutive 4-bit chunks
 - Occurrences of each of the 16 values are counted
 - Compute the a specified minimum entropy metric X
 - The test passes if the value X falls within the specified range
- Monobit (frequency)
 - Count the number of '1' bits in the 20k bit block
 - The test passes if $96554 < N_{\text{ones}} < 10346$
- Runs
 - Define a run as an unbroken sequence of 1s or 0s.
 - Count runs of lengths 1, 2, 3, 4, 5, and 6+
 - The test passes if count values are within specified ranges
- Long runs
 - Similar to the Runs test, but count the number of runs of length >25
 - If any long runs occur then the test fails
- Continuous Run
 - Check for identical 32-bit patterns
 - The test fails if any 32-bit pattern is repeated

NIST SP800-90B also specifies a set of tests. These require two data sets:

- Sequential dataset of 1 million continuous samples
- Restart dataset consisting of 1000 samples from 1000 restarts

Attacks and Countermeasures

We anticipate the following types of physical attacks:

- Temperature Gradient
- Voltage and ground biasing
- Frequency Injection Lock
- Glitching (voltage, laser or EM) to disrupt logic operations such as FIFO push and pop

Each of these attacks will result in a reduction of entropy which must be detectable in the online health tester.

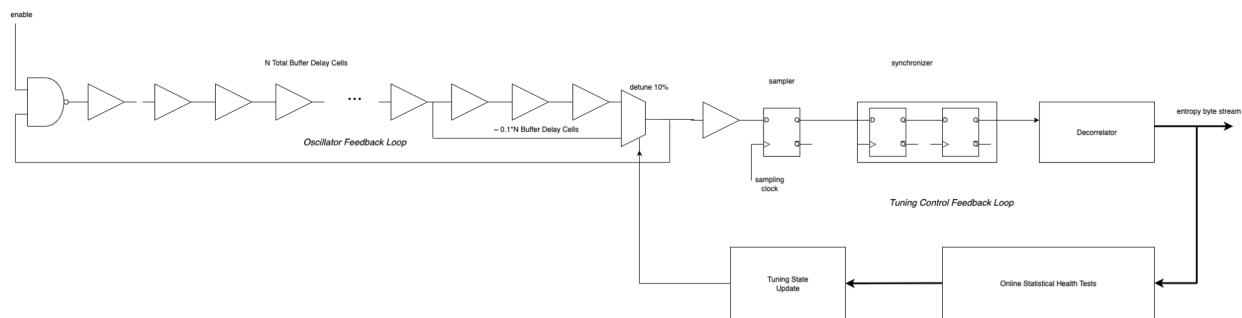
Countermeasures include:

- Detuning individual (or a set) of ring oscillators
- Swapping the noise sampling clock
- Reducing the decorrelator sampling rate (default is every 64 cycles)
- Error detection in the FIFO storage elements as well as pointers (automatic)

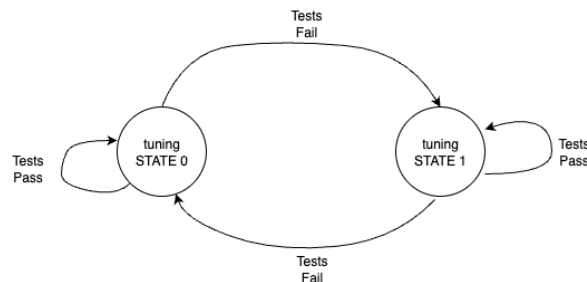
The ring oscillator attack countermeasures are manually controlled by the host. The host can write to various CSRs such as RO detune and sample clock select, to alter the noise source behavior. It can selectively enable or disable each RO.

Automatic countermeasure mode is an optional, best effort feature for this test chip.

The automatic tuning countermeasure is illustrated below. Each of the sampled ring oscillators has an individual online statistical health tester which is fed with the final entropy byte stream. Thresholds are set for each of the tests (repetition, APT and Markov). Test status (pass/fail) is forwarded to the tuning state machine which makes a tuning decision.



In the binary tuning case illustrated here, the decision is simply whether to switch to the alternate frequency or not as shown in the state diagram below. We could conceivably have more tuning states, say four and expand the state machine to four states with states cycling around, but this would need more investigation for a future generation of this component.

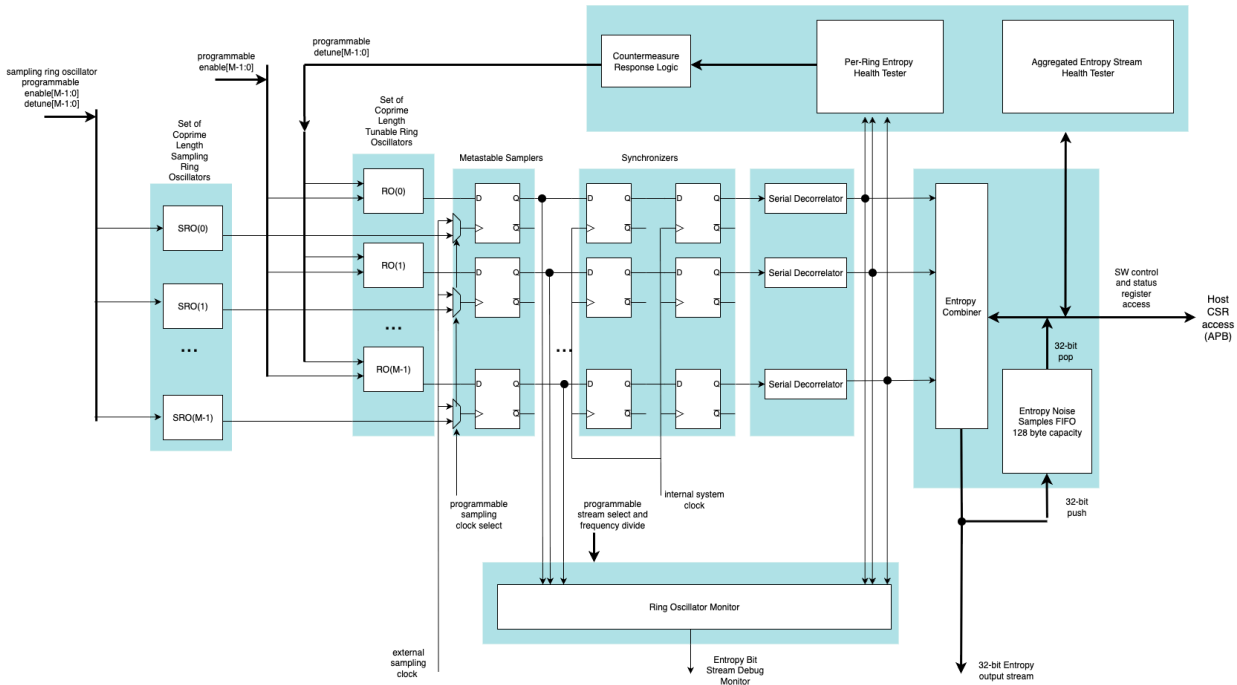


It is important to note that the test fail transitions are not level sensitive but rather sensitive to test failure positive assertion. When a health test failure is asserted, the FSM transitions to the opposite tuning state e.g. from state 0 to 1 or vice versa. We then expect the health tests to start passing again and the failure signal to be deasserted after some number of cycles. We however will stay in this new state until we see a future health test failure assertion, upon which we again switch to the opposite state.

Entropy Generator Core

The core entropy generator is shown below. It consists of a set of ring oscillators of coprime length and corresponding metastable DFF samplers followed by system clock synchronizers and serial decorrelators. The entropy extractor combines the parallel entropy source streams into a single bit stream.

The entropy bit stream is provided as an output and is also fed into the online statistical health monitor where measurements are taken. Measurements are available as host accessible status registers for monitoring. The measurements are fed into the attack countermeasure logic which individually or collectively detunes the entropy sources' ring oscillators.



Functional Design Details

Component Interface

IO Port Signals		Description
Name	Polarity	
clk_i rst_ni	Input Input	Clock Reset (asynchronous, active low)
paddr_i[7:0] pprot_i[2:0] psel_i penable_i pwrite_i pwrdata_i[31:0] prdata_o[31:0] pstrb_i pready_o pslverr_o	Input input Input Input Input Input Output Input Output Output	AMBA APB4 32-bit system bus slave port. Addressing is bitwise.
signal_monitor_o	Output	Ring oscillator monitor output
rosc_sample_clk_i	Input	Ring oscillator sampling clock input
entropy_stream_data_o entropy_stream_vld_o	Output Output	Raw 32-bit entropy output bit stream data and valid
irq_o	Output	Interrupt alarm notification of errors or health test failures

Memory Mapped Configuration and Status Registers (CSR)

The registers will be specified in detail using SystemRDL and a tool will process that description to automatically generate all required formats:

- Verilog hardware
 - CSR register block implementation
 - APB system bus interface to access the CSR block
 - Interrupt logic and interface
- C header file
 - Macros for accessing register fields
- Documentation
 - HTML, PDF etc.

NOTE: This register table is not definitive. Consult the master source SystemRDL file!

OVERVIEW

Component: Entropy Source
Version: 1.0
Address Range: 0x000 - 0x0EC
Bus Width: 32-bit
Reset Type: Asynchronous active-low

The Entropy Source provides a True Random Number Generator (TRNG) based on 12 ring oscillator noise generators. The module includes comprehensive health test monitoring (Repetition, Adaptive Proportion, and Markov tests) and configurable decorrelation processing.

REGISTER SUMMARY TABLE

Offset	Register Name	Access	Description
0x000	COMPONENT_ID	RO	Component Identification
0x004	CTRL	RW	Entropy Source Control
0x008	STATUS	RO	Entropy Source Status
0x00C	DEBUG_CTRL	RW	Debug Control
0x010	INTR_STATUS	RW	Interrupt Status
0x014	INTR_ENABLE	RW	Interrupt Enable
0x018	INTR_TEST	WO	Interrupt Test
0x020	FIFO_CTRL	RW	FIFO Control
0x024	FIFO_STATUS	RO	FIFO Status
0x028	FIFO_RDATA	RO	FIFO Read Data
0x030	HEALTH_TEST_CTRL	RW	Health Test Control
0x038	MARKOV_TEST_PROB_THRESHOLDS	RW	Markov Test Probability Thresholds
0x040	HEALTH_TEST_STATUS	RO	Health Test Status
0x044	REPETITION_TEST_COUNT	RO	Repetition Test Count
0x050	APT_PATTERN_COUNT_1BIT	RO	APT 1-bit Pattern Count

0x054	APT_PATTERN_COUNT_2BIT	RO	APT 2-bit Pattern Count
0x058	APT_PATTERN_COUNT_3BIT	RO	APT 3-bit Pattern Count
0x05C	APT_PATTERN_COUNT_4BIT	RO	APT 4-bit Pattern Count
0x060	APT_PROPORTION_1BIT	RW	APT 1-bit Proportion Limit
0x064	APT_PROPORTION_2BIT	RW	APT 2-bit Proportion Limit
0x068	APT_PROPORTION_3BIT	RW	APT 3-bit Proportion Limit
0x06C	APT_PROPORTION_4BIT	RW	APT 4-bit Proportion Limit
0x080	MARKOV_TEST_COUNTS_0	RO	Markov Test Counts (01/10)
0x084	MARKOV_TEST_COUNTS_1	RO	Markov Test Counts (00/11)
0x088	MARKOV_TEST_PROBABILITIES	RO	Markov Test Probabilities
0x090	RING_OSC_ENABLE	RW	Ring Oscillator Enables
0x094	RING_OSC_TUNE	RW	Ring Oscillator Tune Control
0x098	RING_OSC_CTRL	RW	Ring Oscillator Control
0x0A0	DECORRELATOR_CTRL	RW	Decorrelator Control
0x0A4	DECORRELATOR_MASK	RW	Decorrelator Mask
0x0B0	STARTUP_CTRL	RW	Startup Control
0x0C0	GENERATOR_0_HEALTH_STATUS	RO	Generator 0 Health Status
0x0C4	GENERATOR_1_HEALTH_STATUS	RO	Generator 1 Health Status
0x0C8	GENERATOR_2_HEALTH_STATUS	RO	Generator 2 Health Status
0x0CC	GENERATOR_3_HEALTH_STATUS	RO	Generator 3 Health Status
0x0D0	GENERATOR_4_HEALTH_STATUS	RO	Generator 4 Health Status
0x0D4	GENERATOR_5_HEALTH_STATUS	RO	Generator 5 Health Status
0x0D8	GENERATOR_6_HEALTH_STATUS	RO	Generator 6 Health Status
0x0DC	GENERATOR_7_HEALTH_STATUS	RO	Generator 7 Health Status
0x0E0	GENERATOR_8_HEALTH_STATUS	RO	Generator 8 Health Status
0x0E4	GENERATOR_9_HEALTH_STATUS	RO	Generator 9 Health Status
0x0E8	GENERATOR_10_HEALTH_STATUS	RO	Generator 10 Health Status
0x0EC	GENERATOR_11_HEALTH_STATUS	RO	Generator 11 Health Status

Legend: RO = Read Only, RW = Read/Write, WO = Write Only

DETAILED REGISTER DESCRIPTIONS

COMPONENT_ID (Offset: 0x000)

Access: Read Only
Reset Value: 0x1000_0001

Description:
Component identification register providing version and component information.

Bits	Field Name	Access	Reset	Description
[15:0]	NAME	RO	0x0001	Component ID (0x0001 = Entropy Source)
[27:24]	MINOR_VERSION	RO	0x0	Minor version number
[31:28]	MAJOR_VERSION	RO	0x1	Major version number

CTRL (Offset: 0x004)

Access: Read/Write
Reset Value: 0x0000_0000

Description:
Main control register for entropy source operation.

Bits	Field Name	Access	Reset	Description
[0]	RESET	RW	0x0	Software reset (active high)
[4]	AUTOTUNE_ENABLE	RW	0x0	Enable automatic detuning of ring oscillators on health test failure
[8]	BYPASS_ENTROPY_COMPRESSOR	RW	0x0	Bypass entropy compressor; push all 12 generator bytes

				directly into FIFO
[25:16]	DOWNSAMPLE_RATE	RW	0x0	Output downsample rate applied after decorrelator

STATUS (Offset: 0x008)	
------------------------	--

Access: Read Only
Reset Value: 0x0000_0000

Description:
General status register.

Bits	Field Name	Access	Reset	Description
[0]	RSVD	RO	0x0	Reserved

DEBUG_CTRL (Offset: 0x00C)	
----------------------------	--

Access: Read/Write
Reset Value: 0x0000_0000

Description:
Debug control to monitor internal entropy noise source signals.

Bits	Field Name	Access	Reset	Description
[7:0]	SELECT_SIGNAL	RW	0x0	Select internal signal to monitor
[10:8]	SELECT_FREQ_DIV	RW	0x0	Select frequency divider

INTR_STATUS (Offset: 0x010)	
-----------------------------	--

Access: Read/Write (Write 1 to Clear)

Reset Value: 0x0000_0000

Description:

Interrupt status register. Write 1 to clear individual interrupts.

Bits	Field Name	Access	Reset	Description
[0]	HEALTH_TEST_FAILED	RW1C	0x0	Health test failure interrupt
[4]	FIFO_ERROR	RW1C	0x0	FIFO error interrupt
[8]	FIFO_OVERFLOW	RW1C	0x0	FIFO overflow interrupt
[12]	FIFO_UNDERFLOW	RW1C	0x0	FIFO underflow interrupt

Note: RW1C = Read/Write 1 to Clear (writing 1 clears the bit, writing 0 has no effect)

INTR_ENABLE (Offset: 0x014)	
-----------------------------	--

Access: Read/Write

Reset Value: 0x0000_0000

Description:

Interrupt enable register. Set bit to 1 to enable corresponding interrupt.

Bits	Field Name	Access	Reset	Description
[0]	HEALTH_TEST_FAILED	RW	0x0	Enable health test failure interrupt
[4]	FIFO_ERROR	RW	0x0	Enable FIFO error interrupt
[8]	FIFO_OVERFLOW	RW	0x0	Enable FIFO overflow interrupt
[12]	FIFO_UNDERFLOW	RW	0x0	Enable FIFO underflow interrupt

INTR_TEST (Offset: 0x018)	
---------------------------	--

Access: Write Only

Reset Value: 0x0000_0000

Description:

Interrupt test register. Write 1 to trigger test interrupt.

Bits	Field Name	Access	Reset	Description
[0]	HEALTH_TEST_FAILED	WO	0x0	Trigger health test failure test interrupt
[4]	FIFO_ERROR	WO	0x0	Trigger FIFO error test interrupt
[8]	FIFO_OVERFLOW	WO	0x0	Trigger FIFO overflow test interrupt
[12]	FIFO_UNDERFLOW	WO	0x0	Trigger FIFO underflow test interrupt

Note: Single pulse - automatically clears after one cycle

FIFO_CTRL (Offset: 0x020)	
---------------------------	--

Access: Read/Write
Reset Value: 0x0000_0001

Description:

FIFO control register.

Bits	Field Name	Access	Reset	Description
[0]	ENABLE	RW	0x1	Enable FIFO (1=enabled, 0=frozen)

FIFO_STATUS (Offset: 0x024)	
-----------------------------	--

Access: Read Only
Reset Value: 0x0000_0000

Description:

FIFO status register showing fill level and pointers.

Bits	Field Name	Access	Reset	Description
[6:0]	LEVEL	RO	0x0	FIFO fill level (0-127)
[12:8]	WPTR	RO	0x0	FIFO write pointer
[20:16]	RPTR	RO	0x0	FIFO read pointer

FIFO_RDATA (Offset: 0x028)	
----------------------------	--

Access: Read Only (with side effect)

Reset Value: 0x0000_0000

Description:

FIFO read data register. Reading this register pops one 32-bit word from the FIFO.

Bits	Field Name	Access	Reset	Description
[31:0]	RDATA	RO	0x0	32-bit entropy data from FIFO

Note: Reading this register advances the read pointer

HEALTH_TEST_CTRL (Offset: 0x030)	
----------------------------------	--

Access: Read/Write

Reset Value: 0x0000_0F07

Description:

Health test control register. Configure which tests are enabled and set thresholds.

Bits	Field Name	Access	Reset	Description
[7:0]	ENABLE	RW	0x07	Enable tests: [0] = Repetition [2:1] = Reserved [3] = APT

				[7:4] = Markov (4 tests) Disabling resets test state
[15:8]	REPETITION_LIMIT	RW	0x0F	Repetition test threshold (default=15)

MARKOV_TEST_PROB_THRESHOLDS (Offset: 0x038)	
---	--

Access: Read/Write
Reset Value: 0x6464_6464

Description:
Markov test probability thresholds for transition detection.

Bits	Field Name	Access	Reset	Description
[7:0]	PROB_01_THRESHOLD	RW	100	0→1 transition probability threshold
[15:8]	PROB_10_THRESHOLD	RW	100	1→0 transition probability threshold
[23:16]	PROB_00_THRESHOLD	RW	100	0→0 transition probability threshold
[31:24]	PROB_11_THRESHOLD	RW	100	1→1 transition probability threshold

Note: Threshold values are scaled (0-255 representing 0-100% probability)

HEALTH_TEST_STATUS (Offset: 0x040)	
------------------------------------	--

Access: Read Only
Reset Value: 0x0000_0000

Description:
Consolidated health test status for final 32-bit entropy stream.
0 = Pass, 1 = Fail

Bits	Field Name	Access	Reset	Description
------	------------	--------	-------	-------------

[7:0]	HEALTH_STATUS	RO	0x0	Test status vector: [0] = Repetition [2:1] = Reserved [3] = APT [7:4] = Markov (4 tests)
-------	---------------	----	-----	--

REPETITION_TEST_COUNT (Offset: 0x044)

Access: Read Only
Reset Value: 0x0000_0000

Description:
Current repetition test counter value.

Bits	Field Name	Access	Reset	Description
[7:0]	REPETITION_COUNT	RO	0x0	Current consecutive bit count

APT_PATTERN_COUNT_1BIT / 2BIT / 3BIT / 4BIT (Offsets: 0x050, 0x054, 0x058, 0x05C)
--

Access: Read Only
Reset Value: 0x0000_0000

Description:
Adaptive Proportion Test status for 1-bit, 2-bit, 3-bit, and 4-bit patterns.

Bits	Field Name	Access	Reset	Description
[9:0]	PATTERN_COUNT	RO	0x0	Count of target pattern occurrences
[13:10]	TARGET_PATTERN	RO	0x0	Current target pattern being tracked
[29:20]	SAMPLES_PROCESSED	RO	0x0	Number of samples in current window

APT_PROPORTION_1BIT / 2BIT / 3BIT / 4BIT (Offsets: 0x060, 0x064, 0x068, 0x06C)	
---	--

Access: Read/Write
Reset Value: 0x0000_0258 (600 decimal)

Description:
APT proportion limit thresholds for 1-bit, 2-bit, 3-bit, and 4-bit samples.

Bits	Field Name	Access	Reset	Description
[9:0]	LIMIT	RW	600	Proportion threshold

MARKOV_TEST_COUNTS_0 (Offset: 0x080)	
--------------------------------------	--

Access: Read Only
Reset Value: 0x0000_0000

Description:
Markov test transition counts for 0→1 and 1→0 transitions.

Bits	Field Name	Access	Reset	Description
[15:0]	COUNT_01	RO	0x0	Count of 0→1 transitions
[31:16]	COUNT_10	RO	0x0	Count of 1→0 transitions

MARKOV_TEST_COUNTS_1 (Offset: 0x084)	
--------------------------------------	--

Access: Read Only
Reset Value: 0x0000_0000

Description:
Markov test transition counts for 0→0 and 1→1 transitions.

--	--	--	--	--

Bits	Field Name	Access	Reset	Description
[15:0]	COUNT_00	RO	0x0	Count of 0→0 transitions
[31:16]	COUNT_11	RO	0x0	Count of 1→1 transitions

MARKOV_TEST_PROBABILITIES (Offset: 0x088)	
---	--

Access: Read Only
Reset Value: 0x0000_0000

Description:
Calculated Markov test transition probabilities.

Bits	Field Name	Access	Reset	Description
[7:0]	PROB_01	RO	0x0	Probability of 0→1 transitions
[15:8]	PROB_10	RO	0x0	Probability of 1→0 transitions
[23:16]	PROB_00	RO	0x0	Probability of 0→0 transitions
[31:24]	PROB_11	RO	0x0	Probability of 1→1 transitions

Note: Probability values are scaled (0-255 representing 0-100% probability)

RING_OSC_ENABLE (Offset: 0x090)	
---------------------------------	--

Access: Read/Write
Reset Value: 0x0FFF_0FFF

Description:
Enable control for 12 noise ring oscillators and 12 sampling clock oscillators.

Bits	Field Name	Access	Reset	Description
[11:0]	ENABLE	RW	0xFFFF	Enable for each noise oscillator (bit N enables oscillator N) 1=enabled, 0=disabled

[23:12]	SAMPLE_CLK_ENABLE	RW	0xFFF	Enable for each sampling clock oscillator (bit N+12 enables sampling oscillator N) 1=enabled, 0=disabled
---------	-------------------	----	-------	--

Note: All oscillators enabled by default

RING_OSC_TUNE (Offset: 0x094)	
-------------------------------	--

Access: Read/Write

Reset Value: 0x0000_0000

Description:

Detune control for noise and sampling clock oscillators.

Bits	Field Name	Access	Reset	Description
[11:0]	DETUNE	RW	0x0	Detune each noise oscillator (1=detune, 0=normal) Used with AUTOTUNE_ENABLE=0
[23:12]	SAMPLE_CLK_DETUNE	RW	0x0	Detune each sampling clock oscillator (1=detune, 0=normal) Always manual control

Note: Detuning is disabled by default

RING_OSC_CTRL (Offset: 0x098)	
-------------------------------	--

Access: Read/Write

Reset Value: 0x0000_0FFF

Description:

Sample clock source selection for each oscillator.

Bits	Field Name	Access	Reset	Description
------	------------	--------	-------	-------------

[11:0]	SAMPLE_CLK_SELECT	RW	0xFFFF	Select sample clock source: 1 = Internal ring oscillator 0 = External global clock
--------	-------------------	----	--------	--

Note: Internal ring oscillator clocks selected by default

DECORRELATOR_CTRL (Offset: 0x0A0)	
-----------------------------------	--

Access: Read/Write
Reset Value: 0x0003_F000

Description:
Decorrelator control and sample clock divider configuration.

Bits	Field Name	Access	Reset	Description
[11:0]	BYPASS	RW	0x0	Bypass decorrelator feedback loop per generator (1=bypass, 0=normal)
[31:12]	SAMPLE_CLK_DIV	RW	63	Sample clock divider (actual division = value + 1) Default: 63 → divide by 64

Note: When bypass is enabled, can reduce SAMPLE_CLK_DIV from 63 to 7 (÷8) to capture all raw bits

DECORRELATOR_MASK (Offset: 0x0A4)	
-----------------------------------	--

Access: Read/Write
Reset Value: 0x0000_00FF

Description:
Entropy byte mask for decorrelator output.

Bits	Field Name	Access	Reset	Description
[7:0]	ENTROPY_BYTE_MASK	RW	0xFF	Mask for entropy byte bits (1=enabled, 0=masked)

Note: All bits enabled by default. Masking used for special test cases only.

STARTUP_CTRL (Offset: 0x0B0)

Access: Read/Write
Reset Value: 0x0000_0000

Description:
Startup delay control for noise source stabilization.

Bits	Field Name	Access	Reset	Description
[15:0]	DELAY_CYCLES	RW	0x0	Number of cycles to wait after power-on reset before enabling entropy output to FIFO

Note: Provides time for noise source to stabilize after reset

GENERATOR_x_HEALTH_STATUS (x = 0 to 11) (Offsets: 0x0C0, 0x0C4, 0x0C8, 0x0CC, 0x0D0, 0x0D4, 0x0D8, 0x0DC, 0x0E0, 0x0E4, 0x0E8, 0x0EC)

Access: Read Only
Reset Value: 0x0000_0000

Description:
Individual health test status for each of the 12 entropy generators.

Bits	Field Name	Access	Reset	Description
[7:0]	STATUS	RO	0x0	Health test status byte: [0] = Repetition (0=pass, 1=fail) [2:1] = Reserved [3] = APT (0=pass, 1=fail) [7:4] = Markov Tests (0=pass, 1=fail for each of 4 tests)

Generator Address Mapping:

- GENERATOR_0_HEALTH_STATUS @ 0x0C0 : Generator 0 status
- GENERATOR_1_HEALTH_STATUS @ 0x0C4 : Generator 1 status
- GENERATOR_2_HEALTH_STATUS @ 0x0C8 : Generator 2 status
- GENERATOR_3_HEALTH_STATUS @ 0x0CC : Generator 3 status
- GENERATOR_4_HEALTH_STATUS @ 0x0D0 : Generator 4 status
- GENERATOR_5_HEALTH_STATUS @ 0x0D4 : Generator 5 status
- GENERATOR_6_HEALTH_STATUS @ 0x0D8 : Generator 6 status
- GENERATOR_7_HEALTH_STATUS @ 0x0DC : Generator 7 status
- GENERATOR_8_HEALTH_STATUS @ 0x0E0 : Generator 8 status
- GENERATOR_9_HEALTH_STATUS @ 0x0E4 : Generator 9 status
- GENERATOR_10_HEALTH_STATUS @ 0x0E8 : Generator 10 status
- GENERATOR_11_HEALTH_STATUS @ 0x0EC : Generator 11 status

PROGRAMMING GUIDE

► Basic Initialization Sequence

1. Assert software reset
Write CTRL.RESET = 1
2. Configure health test thresholds
 - HEALTH_TEST_CTRL.REPETITION_LIMIT = desired value (default 15)
 - MARKOV_TEST_PROB_THRESHOLDS = desired thresholds (default 100 each)
 - APT_PROPORTION_xBIT.LIMIT = desired values (default 600)
3. Enable health tests
HEALTH_TEST_CTRL.ENABLE = 0x07 (or desired mask)
4. Configure ring oscillators (if needed)
 - RING_OSC_ENABLE for enable/disable control
 - RING_OSC_TUNE for manual detuning
 - RING_OSC_CTRL for clock source selection
5. Configure decorrelator
DECORRELATOR_CTRL.SAMPLE_CLK_DIV (default 63)
6. Set startup delay (if needed)
STARTUP_CTRL.DELAY_CYCLES
7. Enable interrupts
INTR_ENABLE as desired
8. Release reset
Write CTRL.RESET = 0
9. Enable FIFO
FIFO_CTRL.ENABLE = 1 (enabled by default)

► Reading Entropy

1. Poll FIFO_STATUS.LEVEL to check available data
2. Read FIFO_RDATA to pop 32-bit entropy word
(auto-advances read pointer)
3. Check HEALTH_TEST_STATUS.HEALTH_STATUS to verify data quality
4. Monitor individual generator status via GENERATOR_x_HEALTH_STATUS

► Interrupt Handling

1. Read INTR_STATUS to determine interrupt source

2. Handle interrupt condition:
 - HEALTH_TEST_FAILED: Check health test status registers
 - FIFO_OVERFLOW: Clear FIFO or increase read rate
 - FIFO_UNDERFLOW: Reduce read rate or check entropy generation
 - FIFO_ERROR: Reset and reinitialize
 3. Clear interrupt
 - Write 1 to corresponding INTR_STATUS bit
- Auto-Tune Operation
1. Enable auto-tune
 - CTRL.AUTOTUNE_ENABLE = 1
 2. Monitor GENERATOR_x_HEALTH_STATUS for failures
 3. Hardware automatically detunes failing generators
 4. Check RING_OSC_TUNE.DETUNE to see which generators are detuned
- Debug Mode
1. Configure DEBUG_CTRL.SELECT_SIGNAL to choose internal signal
 2. Configure DEBUG_CTRL.SELECT_FREQ_DIV to set monitoring frequency
 3. Observe selected signal on debug output pin

Ring Oscillators and Noise Sampler

A single ring oscillator is implemented with configuration options:

- Length
- Tap length for detuning

The configuration of each ring instance is specified by a parameter assignment according the table earlier in the document.

All cells are instantiated explicitly using generic standard cell wrappers. Each standard cell wrapper instantiates the particular technology specific cell required. To change technologies we simply swap out the wrapper cell library.

The noise sampler and accompanying synchronizer are non-parameterizable and also instantiate generic standard cells explicitly.

Debug Monitor

The debug monitor is a multiplexor followed by an asynchronous ripple divider. It can be synthesized.

Decorrelator and Extractor

The decorrelator/extractor logic is synthesized and has several configuration options:

- Bypass mode (default: off)
- Length of shift register (default: 29)
- Length of sample word (default: 8)
- Sampling frequency divider (default: 64)
- LFSR mode (default: off)

Health Tester

The health tester modules are all synthesized logic.

Entropy Collection FIFO

The entropy collection FIFO is small at 1024 bits (32 x 16 bits) and is implemented with D flip-flop cells. It can be synthesized. The calculated FIFO push rate is

$200\text{Mbps/decorrelators} * 12 \text{ decorrelators} * (8 \text{ bits out/1 bit in}) * (1 \text{ sample out/64 samples in}) * (4 \text{ bytes out/12 bytes in}) * (1 \text{ FIFO push/32 bits}) = \mathbf{3.125M \text{ FIFO pushes/sec.}}$

To allow for the FIFO push rate to be further lowered, a second down-sampling stage, which is configurable via the Downsample_Rate[9:0] field of the Global Control Register, is added. The default value is 0.

Physical Design Guidelines

The diagram below depicts the approximate cell placement scheme. Figure (a) shows the layout of a single ring oscillator complex. This is the ring oscillator itself together with the sampling flop, synchronizer and decorrelator logic. Note that the delay buffers in the ring oscillator should be abutted to minimize routing loads. The exact placement of the decorrelator logic is not critical.

The following standard cells in the TSMC 12nm library should be used for the ring oscillator complex:

- ND2D0: 2-input NAND2
- BUFFD0: buffer
- MUX2D0: 2-input multiplexor
- DFCNQD0: D flip-flop, asynchronous clear, single output (Q)

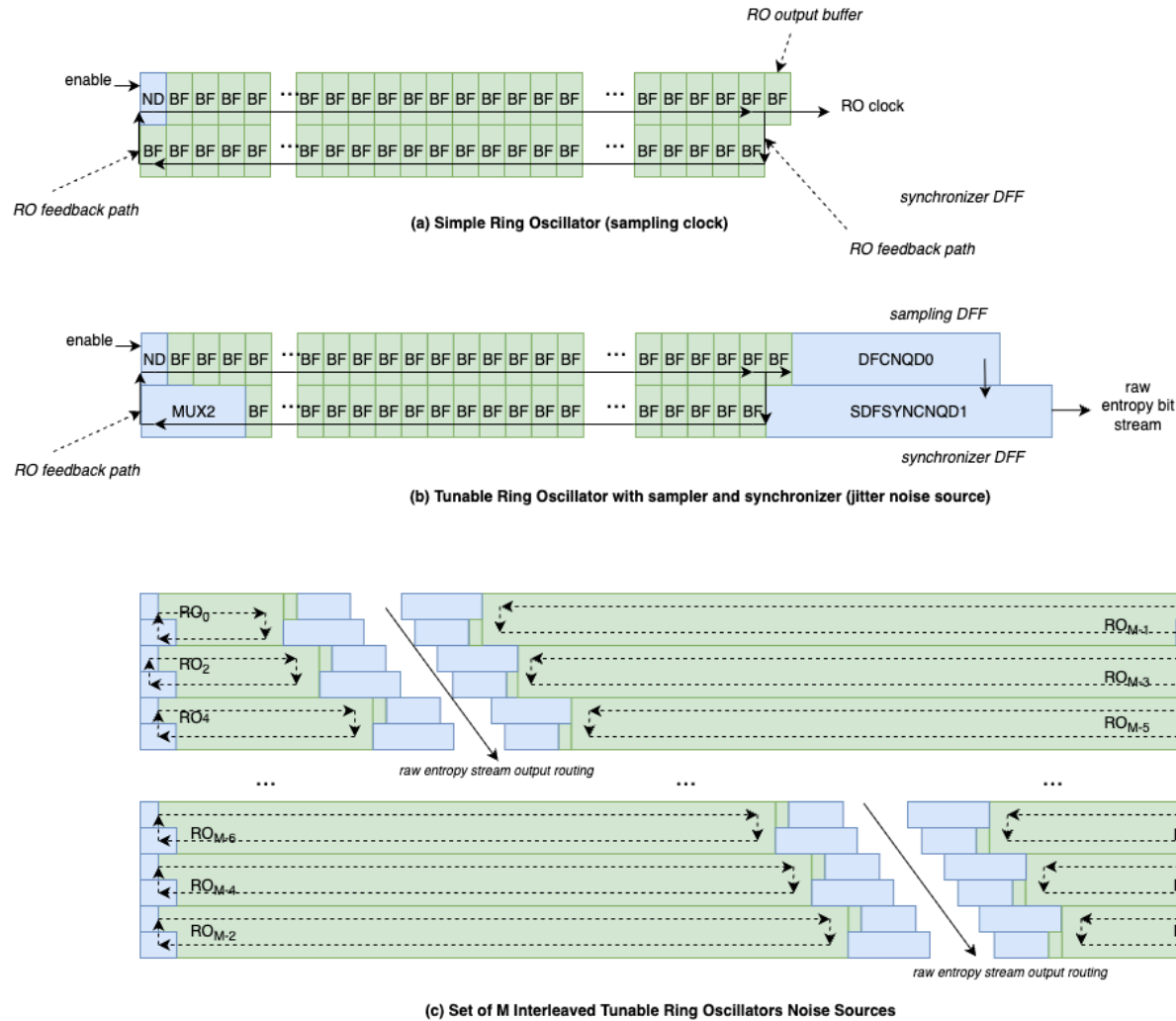
There is some flexibility precisely which versions of these cells should be used, for example with respect to drive strength. What is important is that the chosen cells are instantiated manually with intent and not automatically chosen by logic synthesis.

Other portions of the ring oscillator complex, such as synchronizer and decorrelator logic are more flexible in terms of which cells are chosen. However, that portion of the logic should also be manually designed and placed.

The remaining logic in the entropy source such as the CSR register block, bus interface, statistics monitors etc. are not critical and we expect to be synthesized.

Figure (b) shows the collection of ring oscillator complexes. Note that they are interleaved in pairs so that a constant width is approximately maintained. The side by side oscillators rotate in opposite directions. The oscillators are arranged as mirror images in terms of topology. The decorrelator outputs are routed up or down through the middle of the macro as indicated.

TODO: Create the placement specifications in a spreadsheet (CSV) document. The PD team can automatically translate that format into Synopsys and Cadence placement scripts.



Logic Synthesis and Timing Verification Constraints

Details are still TBD.

Ring oscillators will be manually designed while all remaining logic is synthesized. We may require the following constraint to ensure specially chosen cells are not optimized or replaced with others.

- `set_dont_touch`

The ring oscillators, sampling and synchronization flops are asynchronous so will need constraints for the timing verification flow. For example, combinatorial loops in the ring oscillators will need to be broken with false path directives. Similarly, we'll need to declare false paths on the synchronizer inputs.

- `set_max_delay`
- `set_min_delay`
- `set_false_path`
- `set_disable_timing`

References

1. NIST SP800-90B, "Recommendation for the Entropy Sources Used for Random Bit Generation", <https://csrc.nist.gov/pubs/sp/800/90/b/final>
2. NIST "Random Bit Generation", <https://csrc.nist.gov/projects/random-bit-generation>
3. NIST ESV Entropy Validation Server, Cryptographic Validation Program, <https://csrc.nist.gov/projects/cryptographic-module-validation-program/entropy-validations/esv>
4. NIST SP900-90B Entropy Assessment, https://github.com/usnistgov/SP800-90B_EntropyAssessment
5. BSI, Random Number Generators, https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Informationen-und-Empfehlungen/Kryptografie/Zufallszahlengenerator/zufallszahlengenerator_node.html
6. TSMC 12nm Standard Cell Library, "DB_TCBN12FFCLLBWP20P90CPDLVT_TT0P8V85C.pdf"
7. "µRNG: A 300-950mV, 323 Gbps/W All-Digital Full-Entropy True Random Number Generator in 14nm FinFET CMOS", Intel Corporation July 2016

Appendix: System Context

The Entropy Source is part of the TRNG complex shown in the figure below. The design documented here is a test component. The final component is depicted on the bottom left as "Entropy Noise Source" and will include a SHA256 whitener as well as a streaming output to the DRBG component. The bus slave interface may be either APB or AXI4-lite.

Cryptographic applications of the TRNG, which uses the entropy source include: IV, nonce and key generation. Noise generation for Side Channel Analysis (SCA) and Fault Injection (FI) physical attack countermeasures. Random values for password hash salts, challenge-response protocols.

